DBMS PROJECT BASED LEARNING REPORT

Project 59

Design and Implementation of a Database Management System for Police Complaint and Evidence Tracking

PCETS — Police Complaint & Evidence Tracking System

Submitted by

Name	Arjun Pal
Enrollment No.	25WU0801004
Programme / Section	BCA — TIGERS
Course	Database Management Systems

Technology: MySQL 8.0 · Python 3 (Flask) · October 2026

Table of Contents

Abstract 3

1. Introduction — problem, objectives and scope 3

2. Requirements analysis — users, roles and functions 4

3. Technology stack and system architecture 5

4. Conceptual design — ER model 6

5. Relational schema 8

6. Functional dependencies and normalization (3NF) 8

7. Integrity constraints and business rules 9

8. Triggers, views, procedures and indexing 10

9. SQL queries and reports 11

10. Transactions and application design 11

11. Output screens 13

12. Testing 17

13. Deployment 18

14. Conclusion and future scope 18

References 18

List of abbreviations

Term	Meaning	Term	Meaning
FIR	First Information Report (complaint)	SHO	Station House Officer
IO	Investigating Officer	Malkhana	Police evidence store room
BNS	Bharatiya Nyaya Sanhita (penal code)	NDPS	Narcotic Drugs and Psychotropic Substances Act
ER	Entity–Relationship	FD	Functional dependency
3NF / BCNF	Third / Boyce-Codd Normal Form	RBAC	Role-based access control

Abstract

Police stations still record complaints, investigation diaries, seized evidence and custody hand-overs in separate paper registers. Such registers make it difficult to know which cases are pending, who is working on them and — most critically — where each piece of evidence is and who has held it. A missing or overlapping custody entry can make evidence inadmissible in court.

This project designs and implements **PCETS**, a relational database and web application for police complaint and evidence tracking. The conceptual ER model covers complaints, persons, officers, assignments, investigation activities, evidence items, custody logs, witnesses, charges, court references and status histories. It was converted into **14 relations normalized to 3NF** and implemented in **MySQL 8.0** with primary, foreign, unique, NOT NULL, CHECK and DEFAULT constraints, **17 triggers**, stored procedures, **9 views** and secondary indexes. The five mandatory business rules — unique complaint/evidence IDs, complete custody transfer, valid case status, no overlapping custody and mandatory closure details — are enforced by the database itself. A Python (Flask) application provides complaint registration, officer assignment, activity logging, evidence registration, custody transfer, witness entry, status updates and twelve reports. **43 automated tests** and **26 live integrity scenarios** verify the system, and the application has been deployed to the cloud.

1. Introduction

1.1 Problem identification

In a typical station, the complaint register, case diary, *malkhana* (evidence) register and court diary are maintained separately and by hand. This causes duplicated data (the same person written in many registers), inconsistent updates, no quick way to search or report, and weak chain-of-custody traceability: nothing stops two officers from both being recorded as holding an exhibit, or a hand-over being written without the receiving officer.

1.2 Objectives

- Identify the entities, attributes, relationships and user roles of a police complaint and evidence system.
- Build an ER model and convert it into a relational schema normalized up to Third Normal Form.
- Enforce the business rules with keys, CHECK constraints and triggers so they hold even if the application is bypassed.
- Write SQL for joins, sub-queries, aggregation and views for pending cases, officer load, evidence location, custody history, witnesses, case timeline and closures.
- Develop a validated, transaction-aware Python front end and test it systematically.

1.3 Scope

The system covers one district with several police stations: complaint registration, assignment of officers, the investigation diary, evidence registration and custody, witnesses, accused persons and charges, court references with hearing dates, the case status workflow with a full audit trail, search and reporting. Integration with national systems (CCTNS, e-Courts), payroll and document storage are outside the scope.

2. Requirements Analysis

2.1 Users and roles

Role	Typical user	Permissions
Administrator	District system administrator	All operations, officer management, integrity checks
SHO	Station House Officer (Inspector)	Register/edit complaints, assign and relieve officers, change status, close cases, reports
IO	Investigating Officer (Sub-Inspector)	Log activities, evidence, custody transfers, witnesses, accused, charges, court references, status updates (not closure)
Custodian	Malkhana in-charge (Head Constable)	Register evidence and transfer custody; read-only access to cases

Complainants, accused persons and witnesses are *data subjects*, not users. The application opens directly (no login page); an **"Acting as"** selector chooses the current user, the server checks that role's permissions on every request, and the acting officer is recorded in the audit trail.

2.2 Functional requirements

ID	Requirement
FR-1	Register a complaint with an existing or new complainant and issue a unique complaint number (CMP-YYYY-NNNNN).
FR-2	Assign and relieve officers with roles (Lead, Support, Forensic, Surveillance); only one active lead per case.
FR-3	Log investigation activities (site visit, statement, raid, arrest, CCTV review …) by assigned officers only.
FR-4	Register evidence with a seal number, issue a unique evidence number (EVD-YYYY-NNNNN) and start its chain of custody.
FR-5	Transfer custody with receiving officer, purpose, location, time and seal check.
FR-6	Record witnesses (including protected witnesses) and accused persons.
FR-7	Frame charges (act and section) and add court references and hearing dates.
FR-8	Move a case through a valid status workflow; closing requires closure details; re-opening is allowed.
FR-9	Keep a permanent status history and a complete case timeline.
FR-10	Search and filter complaints, evidence, persons and officers; reports with print and CSV export.

2.3 Non-functional requirements

- **Integrity** — all business rules enforced in the database; custody log and status history are append-only.
- **Atomicity** — multi-step operations run as single transactions.
- **Security** — role-based access, CSRF tokens on every form, only parameterised SQL.
- **Performance** — indexes on frequently searched columns; views pre-join report data.
- **Usability** — responsive interface for desktop and mobile, clear field-level error messages, printable case file.

3. Technology Stack and System Architecture

Layer	Technology	Reason
Database	MySQL 8.0 (InnoDB)	Enforced CHECK constraints (8.0.16+), triggers, window functions, CTEs, transactions
Back end	Python 3, Flask	Lightweight web framework; clear separation of routes and services
DB driver	mysql-connector-python	Official driver; connection pooling and parameterised queries
Front end	HTML (Jinja2), CSS, JavaScript	Responsive UI, light/dark theme, dialogs, charts drawn with SVG/CSS
Testing	Python unittest	Automated tests against a separate test database
Deployment	Clever Cloud / Render, gunicorn	Cloud hosting of the app and MySQL database

3.1 Layered architecture

```
Browser (HTML / CSS / JS)
      │  HTTP
      ▼
Flask routes — complaints · evidence · persons · officers · reports · admin
      │  input validation (validators.py) · role check · CSRF token
      ▼
Service layer — business logic, explicit transactions (START TRANSACTION … COMMIT / ROLLBACK)
      │  parameterised SQL only
      ▼
db.py — connection pool, transaction helper, MySQL error → readable message
      ▼
MySQL 8 — 14 tables · constraints · 17 triggers · 9 views · procedures · indexes
```

Each layer has one job. Routes handle HTTP and permissions, services hold the business rules and transactions, and the database layer is the final authority on integrity. Because the rules live in MySQL, a mistake in the application — or a user typing SQL directly — still cannot break the chain of custody.

3.2 Project structure

```
sql/          00_create_database · 01_schema · 02_triggers_procedures · 03_views
              04_seed_data · 05_report_queries · 06_constraint_tests
police_app/   app.py · auth.py · db.py · validators.py · constants.py · hosted.py
              services/  complaint · evidence · person · officer · report · dashboard · integrity
              routes/    main · complaints · evidence · persons · officers · reports · admin
              templates/ (Jinja2)   static/ (css, js)
tests/        test_app.py (43 automated tests)
setup_db.py · run.py · requirements.txt · gunicorn.conf.py
```

4. Conceptual Design — ER Model

The ER diagram below uses crow's-foot notation. PK marks primary keys, FK foreign keys and UK unique keys.

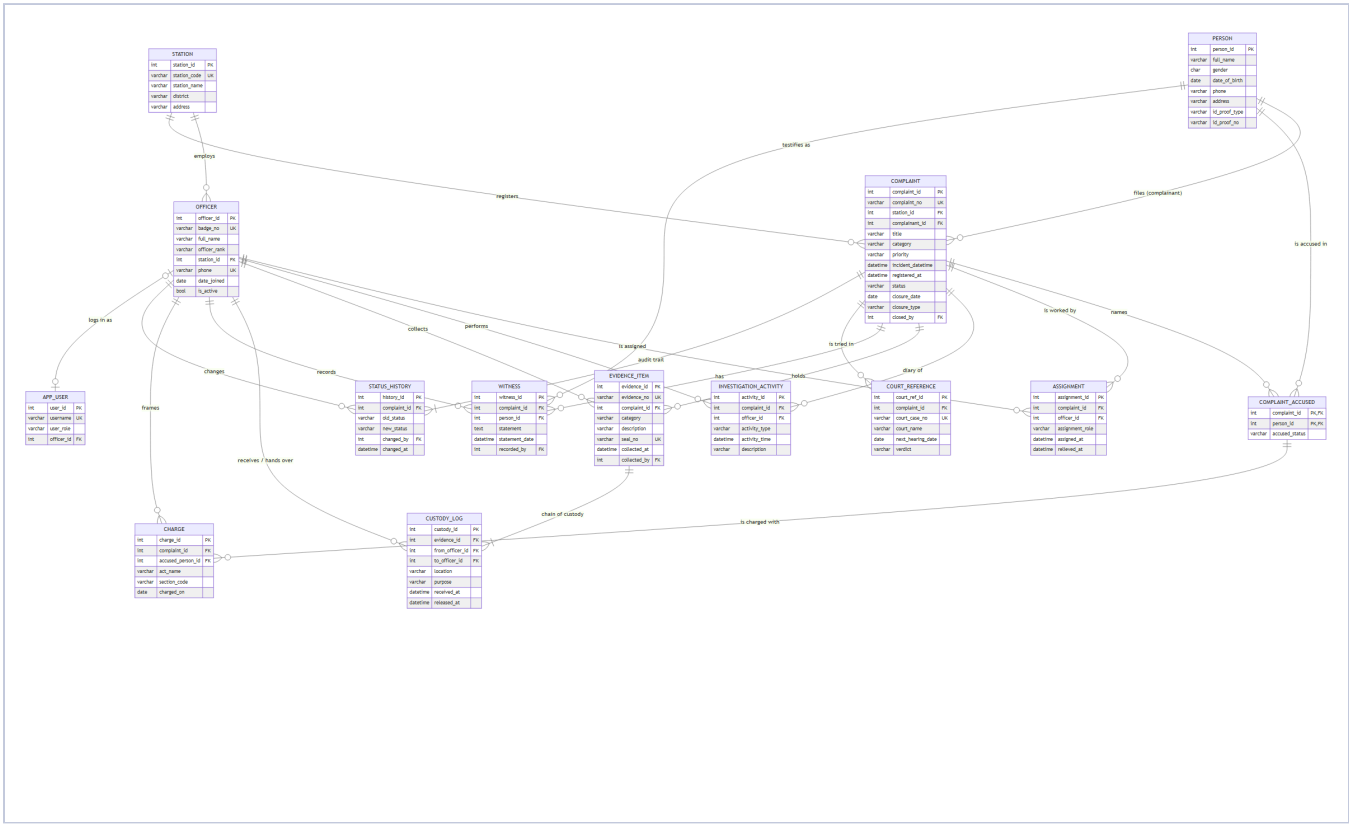


Figure 1. ER diagram of PCETS (14 entities)

> COMPLAINT is the central strong entity. COMPLAINT_ACCUSED, ASSIGNMENT and WITNESS resolve many-to-many relationships; CUSTODY_LOG is a weak-dependent history of EVIDENCE_ITEM; STATUS_HISTORY records every status change.

4.1 Entities

Entity	Description
STATION	Police station or unit that registers complaints and employs officers
OFFICER	Police personnel with badge number, rank and station; may have an application user
PERSON	Any civilian; the same person can be complainant, accused or witness in different cases
COMPLAINT	The case (FIR) with category, priority, incident details, status and closure details
COMPLAINT_ACCUSED	Persons accused in a complaint with their arrest / bail status
ASSIGNMENT	Which officer works on which case, in which role, from when to when
INVESTIGATION_ACTIVITY	Entry in the investigation diary (site visit, raid, arrest ...)
EVIDENCE_ITEM	Seized or collected exhibit belonging to a complaint
CUSTODY_LOG	One custody period of an exhibit: who held it, where, why, from when to when
WITNESS	Statement of a person in a complaint (optionally protected)
CHARGE	Legal section framed against an accused of a complaint
COURT_REFERENCE	Court case number, court, next hearing date, stage and verdict
STATUS_HISTORY	Audit trail of every status change of a complaint
APP_USER	Application user and role

4.2 Relationships and cardinalities

Relationship	Cardinality	Notes
Station employs Officer / registers Complaint	1 : N	every officer belongs to one station
Person files Complaint	1 : N	complainant
Complaint — Person (accused)	M : N	resolved by COMPLAINT_ACCUSED
Complaint — Officer (assignment)	M : N	resolved by ASSIGNMENT with history
Complaint — Person (witness)	M : N	resolved by WITNESS
Complaint has Activities / Evidence / Court references	1 : N	
Evidence has Custody periods	1 : N (≥ 1)	total participation — every item starts with an initial seizure
Complaint_Accused is charged with Charge	1 : N	composite foreign key
Complaint has Status history	1 : N (≥ 1)	first row written on registration

5. Relational Schema

Primary keys are underlined; foreign keys are shown in blue as *column → table*; UK marks unique keys.

STATION(**station_id**, station_code UK, station_name, district, address, phone)

OFFICER(**officer_id**, badge_no UK, full_name, officer_rank, station_id → STATION, phone UK, email UK, date_joined, is_active)

PERSON(**person_id**, full_name, gender, date_of_birth, phone, email, address, id_proof_type, id_proof_no) UK(id_proof_type, id_proof_no)

COMPLAINT(**complaint_id**, complaint_no UK, station_id → STATION, complainant_id → PERSON, title, description, category, priority, incident_datetime, incident_location, registered_at, registered_by → OFFICER, status, closure_date, closure_type, closure_remarks, closed_by → OFFICER)

COMPLAINT_ACCUSED(**complaint_id → COMPLAINT, person_id → PERSON**, accused_status, added_at, remarks)

ASSIGNMENT(**assignment_id**, complaint_id → COMPLAINT, officer_id → OFFICER, assignment_role, assigned_at, relieved_at, assigned_by → OFFICER, remarks)

INVESTIGATION_ACTIVITY(**activity_id**, complaint_id → COMPLAINT, officer_id → OFFICER, activity_type, activity_time, location, description, outcome)

EVIDENCE_ITEM(**evidence_id**, evidence_no UK, complaint_id → COMPLAINT, category, description, quantity, item_condition, seal_no UK, collected_at, collected_by → OFFICER, collected_from)

CUSTODY_LOG(**custody_id**, evidence_id → EVIDENCE_ITEM, from_officer_id → OFFICER, to_officer_id → OFFICER, location, purpose, received_at, released_at, seal_intact, remarks)

WITNESS(**witness_id**, complaint_id → COMPLAINT, person_id → PERSON, statement, statement_date, recorded_by → OFFICER, is_protected)

CHARGE(**charge_id**, (complaint_id, accused_person_id) → COMPLAINT_ACCUSED, act_name, section_code, offence_description, charged_on, charged_by → OFFICER)

COURT_REFERENCE(**court_ref_id**, complaint_id → COMPLAINT, court_case_no UK, court_name, court_type, filing_date, next_hearing_date, judge_name, case_stage, verdict)

STATUS_HISTORY(**history_id**, complaint_id → COMPLAINT, old_status, new_status, changed_by → OFFICER, changed_at, remarks)

APP_USER(**user_id**, username UK, password_hash, user_role, officer_id → OFFICER UK, is_active)

6. Functional Dependencies and Normalization

6.1 Key functional dependencies

Relation	Functional dependencies
OFFICER	officer_id → badge_no, full_name, officer_rank, station_id, phone, …; badge_no → officer_id
COMPLAINT	complaint_id → complaint_no, station_id, complainant_id, title, …, status, closure_*; complaint_no → complaint_id
COMPLAINT_ACCUSED	(complaint_id, person_id) → accused_status, added_at, remarks
EVIDENCE_ITEM	evidence_id → evidence_no, complaint_id, category, …; evidence_no → evidence_id; seal_no → evidence_id
CUSTODY_LOG	custody_id → evidence_id, from/to officer, location, purpose, received_at, released_at; (evidence_id, received_at) → custody_id
WITNESS	witness_id → …; (complaint_id, person_id) → witness_id
CHARGE	charge_id → …; (complaint_id, accused_person_id, act_name, section_code) → charge_id

6.2 Normal forms

- **First Normal Form:** every attribute is atomic. Multiple accused, witnesses, charges, custody steps and status changes are stored as rows of their own tables, never as lists inside a column.
- **Second Normal Form:** the only composite primary key is COMPLAINT_ACCUSED(complaint_id, person_id). Its non-key attributes describe the pair, not either part alone, so there are no partial dependencies. All other tables have single-column keys.
- **Third Normal Form:** in every relation the determinant of each non-trivial FD is a candidate key, so there are no transitive dependencies. The schema is in fact in **BCNF**.

6.3 Normalization decisions

Register-style design	Problem	Normalized decision
evidence(current_holder, current_location, status)	Transitive dependency on the latest custody row; moving an item means updating two places	Holder, location and status are **derived** from the open CUSTODY_LOG row by the view `v_evidence_location`
complaint(officer_name, officer_rank)	officer_id → name, rank is transitive; only one officer per case	Separate ASSIGNMENT table (M : N with history)
complaint(complainant_name, phone, address)	Same person repeated across cases; update anomalies	One PERSON table referenced by complaint, accused and witness
complaint(accused1, accused2, sections …)	Repeating groups violate 1NF	COMPLAINT_ACCUSED and CHARGE tables
charge with two independent foreign keys	Could charge someone who is not an accused of that case	Composite FK to COMPLAINT_ACCUSED
complaint(status) only	History of status changes is lost	STATUS_HISTORY written on every change

7. Integrity Constraints and Business Rules

Business rule	How MySQL enforces it
Unique complaint / evidence IDs	UNIQUE keys on `complaint_no` and `evidence_no`; numbers generated by triggers in the format CMP-YYYY-NNNNN / EVD-YYYY-NNNNN; CHECK with REGEXP on the format; numbers cannot be changed
Complete custody transfer	NOT NULL receiving officer, location, purpose and time; CHECK that every non-seizure entry names the handing officer; trigger: handing officer must be the current holder and the new period must start exactly when the previous one ended (no gap)
Valid case status	CHECK limits status to 5 values; trigger enforces the workflow REGISTERED → UNDER_INVESTIGATION → CHARGE_SHEETED → IN_COURT → CLOSED (and re-open) with preconditions: officer assigned, at least one charge, a court reference
No overlapping custody	UNIQUE index on a generated column allows only one open custody period per exhibit; trigger rejects any overlapping time interval; custody log is append-only and cannot be deleted
Closure details required	CHECK: status CLOSED ⇔ closure date, type, remarks and closing officer all present; closure date cannot precede registration

7.1 Additional constraints

- One active LEAD officer per case and no duplicate active assignment (UNIQUE indexes on generated columns).
- Inactive officers cannot be assigned or receive custody; activities only by officers assigned at that time.
- The accused cannot be a witness or complainant in the same case; charges only against accused of that case.
- Court reference only after charge-sheet; a verdict only when the court case is disposed.
- Formats of Aadhaar, PAN, Voter ID, passport, driving licence, mobile and badge numbers checked with CHECK + REGEXP.

8. Triggers, Views, Procedures and Indexing

8.1 Triggers (17)

Trigger	Purpose
`trg_complaint_bi / _ai`	Generate the complaint number, start in REGISTERED, write the first history row
`trg_complaint_bu / _au`	Status workflow and preconditions; write status history with the acting officer
`trg_custody_bi / _bu / _bd`	Complete-transfer, no-gap and no-overlap checks; append-only log; no deletes
`trg_evidence_bi`	Generate the evidence number; case open; collected after the incident
`trg_assignment_bi, trg_activity_bi`	Active officers only; activities only by assigned officers
`trg_witness_bi, trg_accused_bi, trg_charge_bi, trg_court_bi`	Role conflicts and workflow preconditions
`trg_complaint_bd, trg_history_bu / _bd`	Complaints cannot be deleted; the audit trail cannot be changed

Example: `trg_custody_bi` counts existing custody periods of the same exhibit whose time interval intersects the new one and raises `SIGNAL SQLSTATE '45000'` ("Overlapping custody period") if any exist.

8.2 Views (9)

View	Report	Technique
`v_pending_cases`	Pending cases	JOIN, LEFT JOIN, correlated sub-queries
`v_officer_load`	Officer load	COUNT(DISTINCT CASE ...), GROUP BY
`v_evidence_location`	Evidence location	Join on the open custody row; derived status
`v_custody_history`	Custody history	ROW_NUMBER() window function
`v_case_witnesses`, `v_case_timeline`, `v_closed_cases`	Witnesses, timeline, closures	Multi-table JOIN, UNION ALL of 8 event sources, DATEDIFF

8.3 Procedures and indexes

Procedures `sp_transfer_custody` and `sp_close_complaint` run atomically with a rollback handler. Fifteen secondary indexes (e.g. `complaint(status, registered_at)`, `custody_log(evidence_id, received_at)`, a FULLTEXT index on complaint text) speed up searches and reports.

9. SQL Queries and Reports

The file `05_report_queries.sql` contains 25 queries demonstrating INNER and LEFT joins, GROUP BY / HAVING / ROLLUP, conditional aggregation, nested and correlated sub-queries, EXISTS / NOT EXISTS, derived tables, CTEs and window functions. Examples:

```
-- Officers handling more active cases than average (nested sub-query)
SELECT full_name, active_cases FROM v_officer_load
 WHERE active_cases > (SELECT AVG(active_cases) FROM v_officer_load WHERE is_active = 1);

-- Cases per station by status with closure rate (aggregation)
SELECT s.station_name, COUNT(*) AS total,
       SUM(c.status = 'CLOSED') AS closed,
       ROUND(100 * SUM(c.status = 'CLOSED') / COUNT(*), 1) AS closure_rate_pct
  FROM complaint c JOIN station s ON s.station_id = c.station_id
 GROUP BY s.station_name;

-- Open cases with no activity in the last 30 days (NOT EXISTS)
SELECT c.complaint_no, c.title FROM complaint c
 WHERE c.status <> 'CLOSED'
   AND NOT EXISTS (SELECT 1 FROM investigation_activity a
                   WHERE a.complaint_id = c.complaint_id
                     AND a.activity_time >= CURDATE() - INTERVAL 30 DAY);

-- Chain-of-custody integrity audit (window function LAG)
WITH chain AS (
  SELECT evidence_id, received_at, from_officer_id,
         LAG(released_at)   OVER (PARTITION BY evidence_id ORDER BY received_at) AS prev_released,
         LAG(to_officer_id) OVER (PARTITION BY evidence_id ORDER BY received_at) AS prev_holder
    FROM custody_log)
SELECT COUNT(*) AS broken_links FROM chain
 WHERE NOT ((prev_released IS NULL AND from_officer_id IS NULL)
         OR (prev_released = received_at AND prev_holder = from_officer_id));
```

The application offers twelve reports built on these views: pending cases, officer workload, evidence location, custody history, witness register, case timeline, closures, station summary, category analysis, upcoming hearings, stale cases and the custody integrity audit. Each report can be filtered, printed and exported to CSV.

10. Transactions and Application Design

10.1 Transactions

Operation	Statements in one transaction
Register complaint	new person (optional) → complaint → first history row → optional lead assignment and status change
Register evidence	evidence item → initial-seizure custody row → optional deposit with the custodian
Custody transfer	**SELECT … FOR UPDATE** on the open custody row → close it → open the next period
Close case	lock the complaint → update status and closure details → relieve all active officers

Any error rolls the whole operation back. The row lock prevents two custodians from transferring the same exhibit at the same moment.

10.2 Application modules

Module	Features
Dashboard	Key figures, case-status chart, registrations vs closures, officer workload, items needing attention, recent activity
Complaints	Search, filters, sorting, pagination; registration; case page with status stepper and tabs for officers, investigation diary, evidence, accused and witnesses, charges and court, timeline and status audit; printable case file
Evidence & custody	Malkhana register, evidence registration, visual chain of custody, custody-transfer dialog, printable custody sheet
Persons / Officers	Create, read, update and delete with referential protection; involvement across cases; workload profile
Reports	Twelve parameterised reports with charts, print and CSV export
Integrity lab	Runs 26 invalid SQL statements against MySQL (always rolled back) and shows the database error for each rule
Database explorer	Tables, columns, constraints, relationships, triggers and indexes read from information_schema

10.3 Validation and error handling

- **Two layers of validation:** the application checks required fields, lengths, formats, allowed values and that dates are not in the future, then MySQL re-checks everything.
- Database errors are translated into readable messages — duplicate key, CHECK violation, foreign key, trigger message — and the form is shown again with the user's input.
- Friendly 403, 404 and 500 pages; unexpected errors are logged to a file.

10.4 Sample data

The database is populated with realistic data for four Pune police stations: 16 officers, 40 persons, 20 complaints at every stage of the workflow (robbery, burglary, cyber fraud, narcotics, missing person, domestic violence …), 22 evidence items with 56 custody periods, 56 investigation activities, 11 witnesses, 10 charges, 3 court references and 52 status-history rows. All rows were inserted through the triggers, which shows that the business rules are mutually consistent.

11. Output Screens



Figure 2. Dashboard with key figures and charts

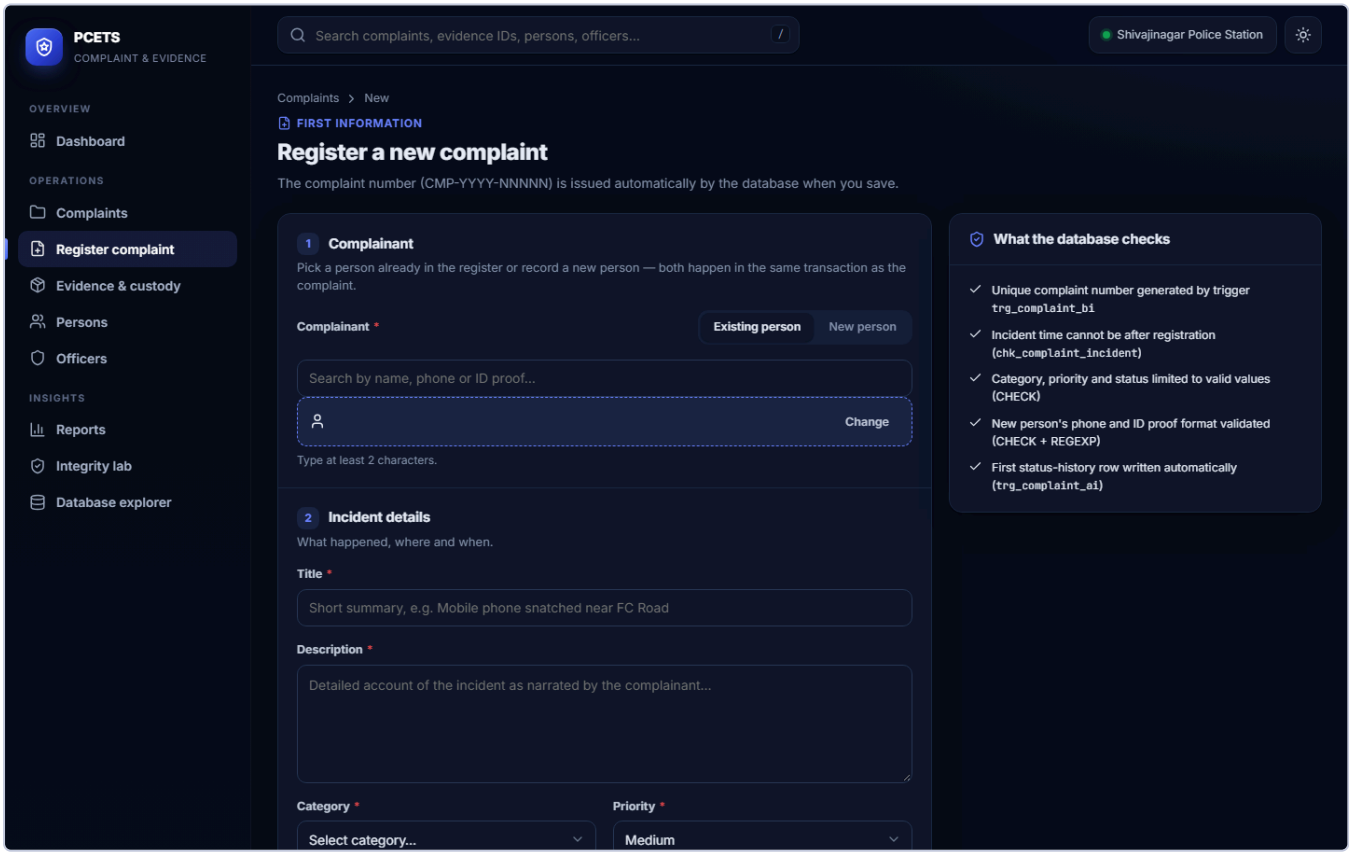


Figure 3. Complaint registration form

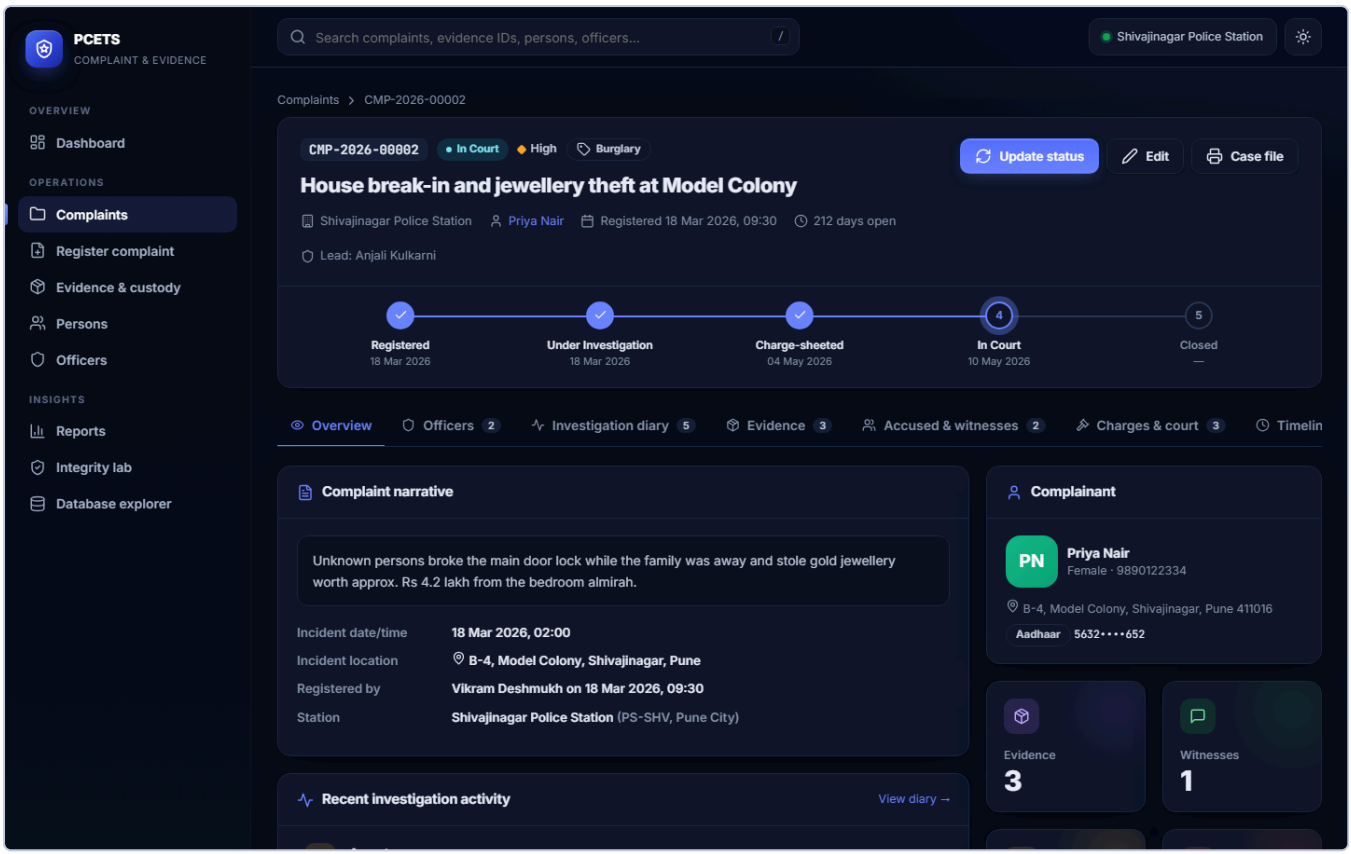


Figure 4. Case page with status workflow stepper

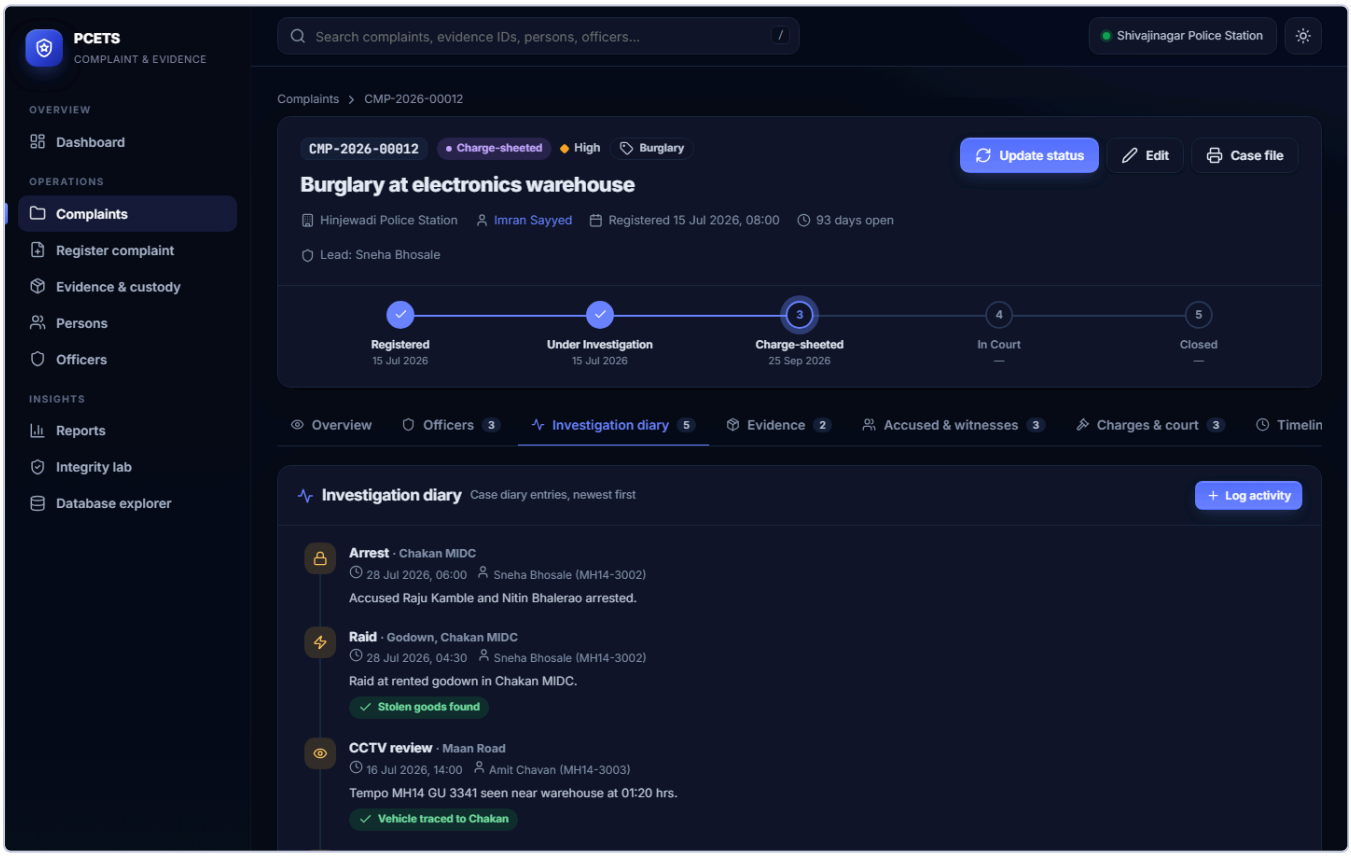


Figure 5. Investigation diary (activity log)

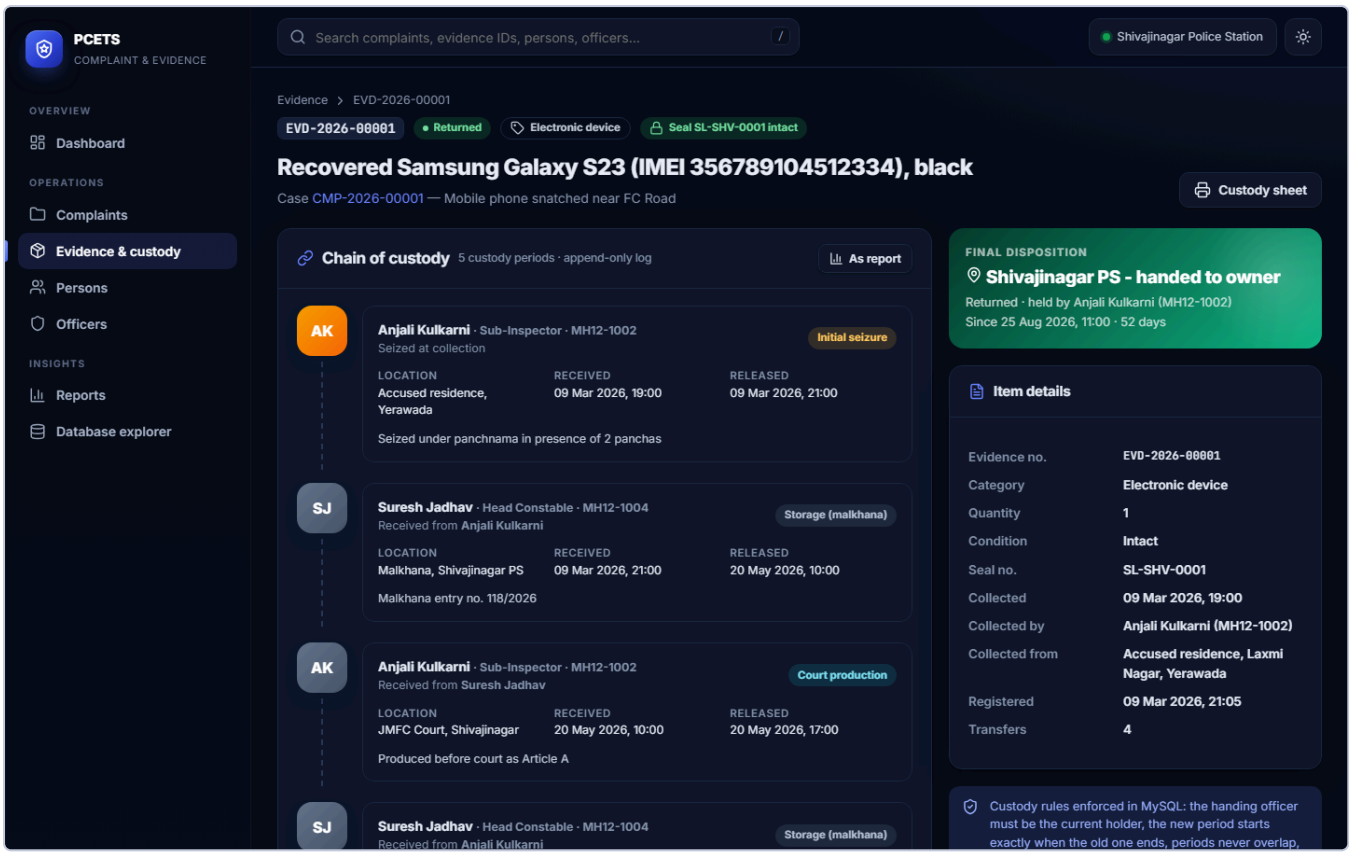


Figure 6. Chain of custody of an exhibit returned to its owner

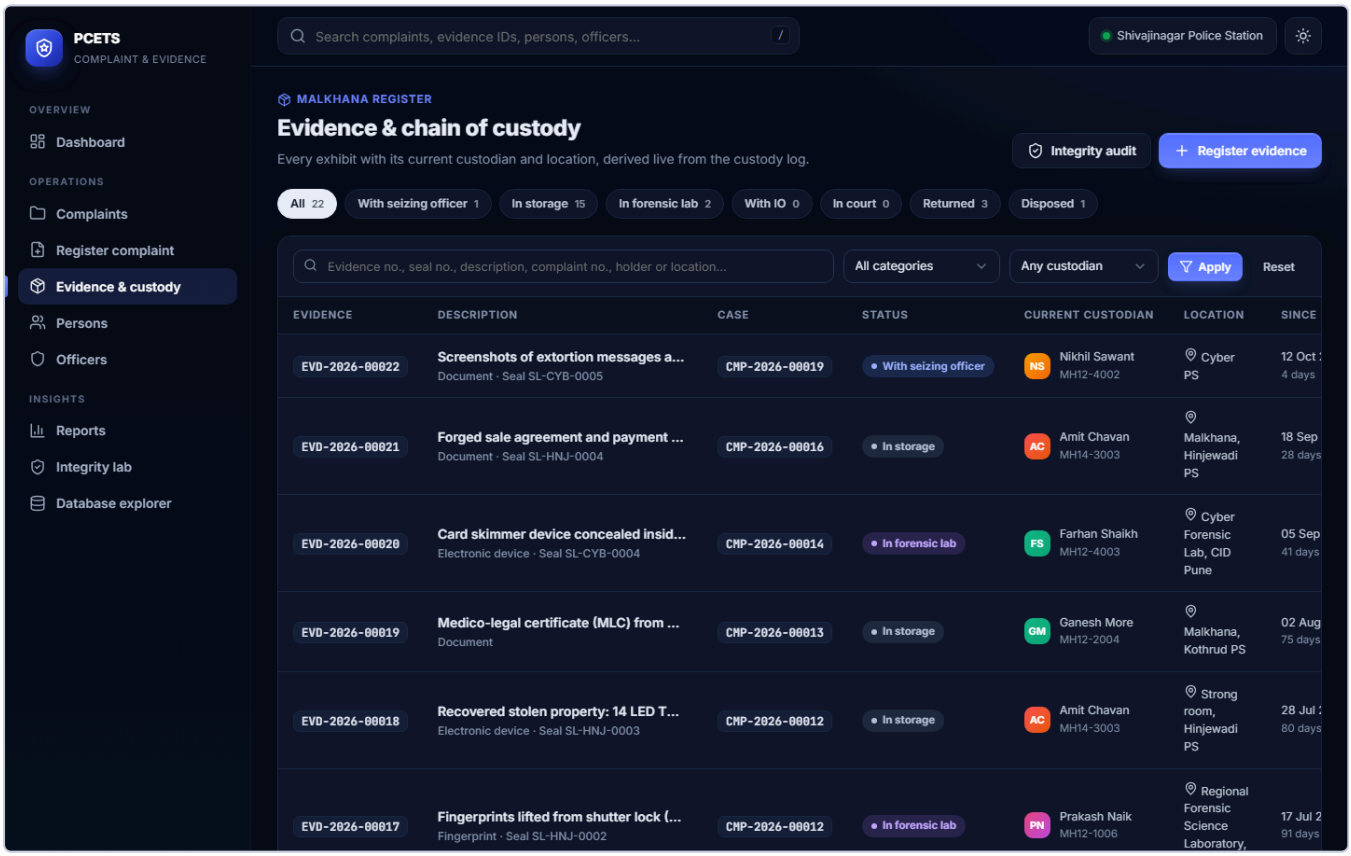


Figure 7. Evidence register with current location and status

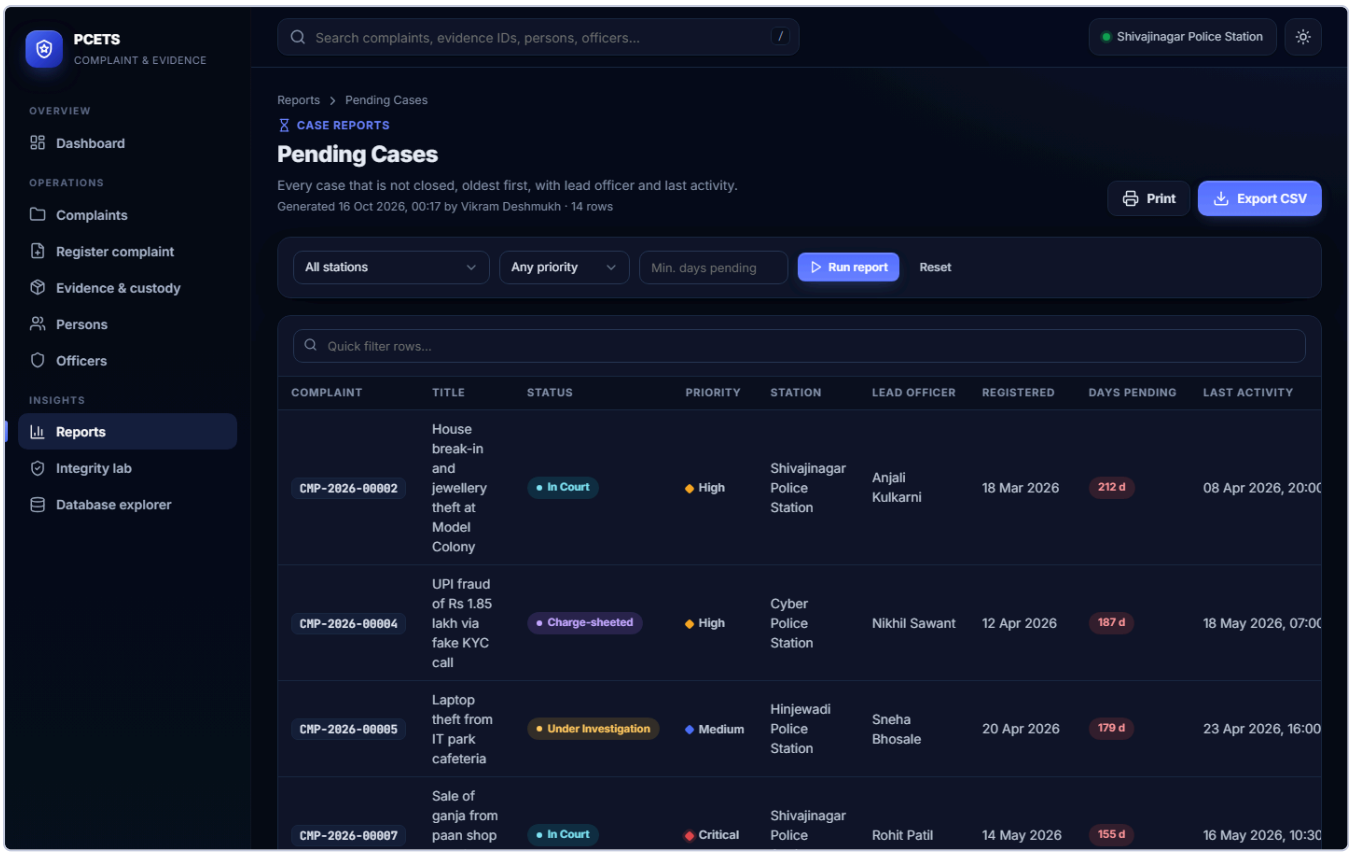


Figure 8. Report — pending cases

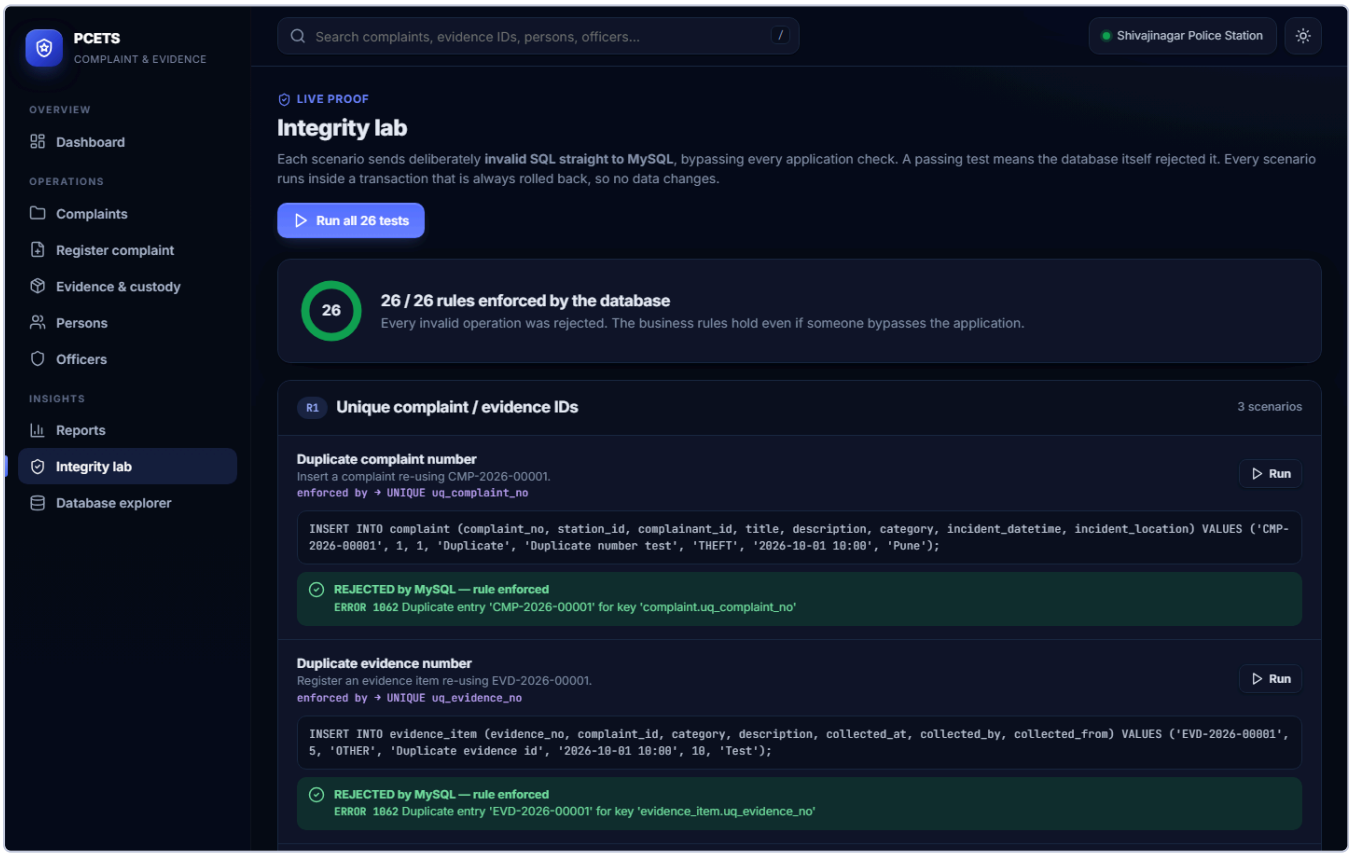


Figure 9. Integrity lab — MySQL rejects every invalid operation

12. Testing

Testing was carried out at three levels: **43 automated functional tests** (Python unittest, run against a separate test database rebuilt for every run), **28 negative SQL tests** in `06_constraint_tests.sql` (each statement must fail), and **26 live integrity scenarios** in the application. All automated tests passed.

ID	Test scenario	Expected result	Result
TC-04	Custodian opens “Register complaint”	Access denied (HTTP 403)	**Pass**
TC-06	Register a complaint with a new complainant	CMP number issued, status REGISTERED, history row	**Pass**
TC-08	Empty title, invalid category, future incident date	Field errors, nothing saved	**Pass**
TC-10	Valid new person but complaint fails in the database	Whole transaction rolled back	**Pass**
TC-12	Assign a second LEAD officer	Rejected — case already has a lead	**Pass**
TC-16	Officer not on the case logs an activity	Rejected	**Pass**
TC-18	Register evidence and deposit it	EVD number, two custody periods with no gap	**Pass**
TC-19	Second exhibit with the same seal number	Rejected (UNIQUE)	**Pass**
TC-20	Custody transfer to the forensic lab	Old period closed, new one opened, status IN_LAB	**Pass**
TC-21	Back-dated custody transfer	Rejected; chain unchanged	**Pass**
TC-23	Transfer an exhibit already returned to its owner	Rejected — chain is final	**Pass**
TC-26	Add a witness as accused in the same case	Rejected	**Pass**
TC-28	Charge-sheet a case without any charge	Rejected	**Pass**
TC-32	Close a case without closure details	Rejected; status unchanged	**Pass**
TC-34	Close with all details	CLOSED, officers relieved in the same transaction	**Pass**
TC-38	Delete a person who filed a complaint	Blocked by foreign key, readable message	**Pass**
TC-43	LAG() audit of the whole custody log	0 broken links	**Pass**

The full list of 43 test cases is in `docs/TEST_CASES.md`. In the Integrity Lab, all 26 scenarios — duplicate IDs, incomplete transfers, invalid status, overlapping custody, missing closure details and others — were rejected by MySQL with an error code.

13. Deployment

The application is deployed to the cloud with a managed MySQL database on Clever Cloud; the web app runs under gunicorn (Clever Cloud and Render). The free MySQL plan does not allow triggers (binary logging without SUPER privilege) and allows only five connections, so the application has a **hosted mode**: when no triggers exist it issues the CMP/EVD numbers, writes the status history and checks the trigger rules itself, and it opens one database connection per request. Keys, CHECK constraints and foreign keys are still enforced by MySQL. On a local MySQL installation the full trigger set is active.

Running locally

```
pip install -r requirements.txt
copy config.example.ini config.ini     (set the MySQL password)
python setup_db.py                      creates the database with schema, triggers, views and data
python run.py                           open http://127.0.0.1:5000
python -m unittest -v tests.test_app    automated tests
```

14. Conclusion and Future Scope

PCETS replaces separate station registers with one normalized database whose rules cannot be bypassed. Complaint and evidence numbers are unique, every custody hand-over is complete and gap-free, custody periods never overlap, case status follows a valid workflow with a permanent audit trail, and a case cannot be closed without its closure details. The Python application provides every required function — complaint registration, officer assignment, activity logging, evidence registration, custody transfer, witness entry, status update and reporting — with validation, transactions and role-based access. The project gave practical experience in ER modelling, normalization, constraint design, triggers, views, transactions, indexing and systematic testing.

Future scope

- Upload photos and documents with each exhibit and store a hash to prove the file is unaltered.
- QR codes on evidence tags scanned at every hand-over.
- SMS / e-mail reminders for hearing dates and exhibits staying too long in the forensic lab.
- Integration with CCTNS and e-Courts; a proper login screen with two-factor authentication for real deployment.

References

1. A. Silberschatz, H. F. Korth, S. Sudarshan — *Database System Concepts*, 7th ed., McGraw-Hill.
2. R. Elmasri, S. B. Navathe — *Fundamentals of Database Systems*, 7th ed., Pearson.
3. MySQL 8.0 Reference Manual — CHECK constraints, triggers, window functions. dev.mysql.com/doc
4. Flask documentation. flask.palletsprojects.com
5. MySQL Connector/Python Developer Guide. dev.mysql.com/doc/connector-python